DD2459 Software Reliability 2023
Take-home Exam
Tuesday 7 March, 15.00 – Thursday 9 March, 12.00 midday
(Version 1.0)

Instructions.
Please read the following instructions carefully.

(1) Only hard-copy exam submissions are accepted. Clearly mark at the top of each sheet of paper you use: (a) your name, (b) the page number.

(2) On your front page indicate: (a) how many pages are contained in your work in total, (b) your name (c) your personal or KTH number, (d) your e-mail address (in case I need to contact you)

(3) There are two ways to submit your work. (**a**) Your work may be handed in personally to EECS studentexpedition, E building, level 4, Lindstedsvägen 3, no later than Thursday 9 March 2023 at 12.00 midday. After this time, it will be marked as late, and marks will be subtracted. (**b**) If you are unable to reach the studentexpedition yourself (for example if you are at work) you may post your manuscript to: Karl Meinke, EECS School, KTH Kungliga Tekniska Högskolan, 100 44 Stockholm, Sweden. The date on the postmark will be taken as the date of your submission. The deadline of March 9, 2023 applies to all manuscripts submitted by post.

(4) If manuscripts are submitted in any other place or by any other means than those described in Part (3) then the examiner and EECS School cannot be held responsible in the case that manuscripts are lost. In the case of postal submission, it is strongly recommended that you keep a digital copy or photocopy in case of postal loss. KTH cannot be held responsible for loss in any national postal system.

(5) If you have any questions about the exam (for example, if you do not understand a question) you may call the examiner on 08 790 6337. Please do not call before 10.00 am or after 5.00pm, and be aware that I may be in a meeting! You can also e-mail the examiner at karlm@kth.se.

(6) I will publish any typographical errors and corrections that become known during the exam on the course exam web page and notify you with Canvas. Therefore, you should check back regularly on the course exam page for updates that will have new version numbers 1.x.

(7) You may use your course notes, books and the internet. However, all material you submit must be your own. (a) You are not allowed to discuss your answers with anyone else until the exam is finished. (b) You are not allowed to copy anyone else's work. (c) *By handing in your manuscript you are declaring that you have read and abide by all the rules on this cover page*. (d) In the case that cheating is suspected, disciplinary action will be taken.

(8) Hand-written answers must be written clearly. No marks will be awarded for work that I cannot read. You can write your answers in Swedish or in English. You can submit a hand-written manuscript or a typed manuscript or any combination of both.

Please turn over.

NOTE: For a full score of 80 points you should answer all 4 questions below.

Question 1. (Total 30 points)

Write appropriate pre- and post-conditions for the following mathematical operations using the JML specification language (i.e. write *requires-ensures* conditions).
You should try to avoid under-specification, i.e. write all the constraints necessary for each condition and do not omit any relevant constraints.
If you are unsure how any mathematical operation/method is defined you may look it up online or in a book.
You do not have to provide Java code for any of the operations, only JML.

(i) **(5 points)** An *identity matrix generator* for 2-dimensional floating-point valued matrices*:*

```
float[][] identity(int n)
```

Input: A non-zero integer size parameter `n`.
Output: The identity matrix of size `n` (commonly termed I_n in maths books).

(ii) **(10 points)** A *matrix inversion function* for floating point valued matrices*:*

```
float[][]  invert(float[][] A)
```

Input: A matrix `A` of floating-point values.

Output: The matrix inverse of `A`.
Hint : You may use the method

```
float[][] identity(int n)
```

of part (i) in your answer. You should avoid any use of determinants which are very complicated to define for `n` > 3.

(iii) **(15 points)** Comment on the practical usefulness of your JML precondition in part (ii) to generate test cases for a Java implementation of matrix inversion. Can you see any possibility to improve your JML answer to make a more efficient solution for test case generation? Write out a test script based on your more efficient proposal that combines random testing with your JML requirement model to achieve both: **(a)** automated valid test case generation and **(b)** automated verdict construction (the oracle step).

Question 2. **(Total 10 points)**

(i) **(6 points)** Write down two different metamorphic equations that should be satisfied by any function:

```
boolean contains(int[] myIntList, int key)
```

with

Input: A one-dimensional array `myIntList` of `int` and an integer `key` .
Output: `true` if `myIntList` contains `key` otherwise `false`.

Clearly define the data transformation T used on the input for each of your two equations.

(ii) **(4 points)** Starting from one concrete seed test case (i.e. input array values), write down as many different test cases as you can generate using the two data transformations T, T' that you defined in your answer to part (i). Is there any finite limit on the number of different test cases you can generate in either case? If so, explain what is the limiting factor.

Question 3. **(Total 15 points)**

Figure 1 shows a 4-way road junction where a side road (heading north and south) cuts across a main road (heading east and west). The main road usually carries more traffic than the side road. Therefore, a traffic light is installed at the junction to allow all traffic to access the junction in a controlled manner. The requirements on the traffic light (from the perspective of the main road drivers) are defined by:

(1) The main road lights (facing east and west) stay green unless a car is sensed on the side road, or a pedestrian tries to cross the main road by pushing a request button.
(2) From green, the main road lights turn amber for two time-units
(3) From amber the main road lights turn red.
(4) The main road lights are red for at least two time-units, and remain red so long as a car is sensed on the side road, or a pedestrian has requested to cross the main road.
(5) From red, the main road lights turn red&amber (both colours) for one time-unit
(6) From red&amber the main road lights turn green.

(Please turn over)

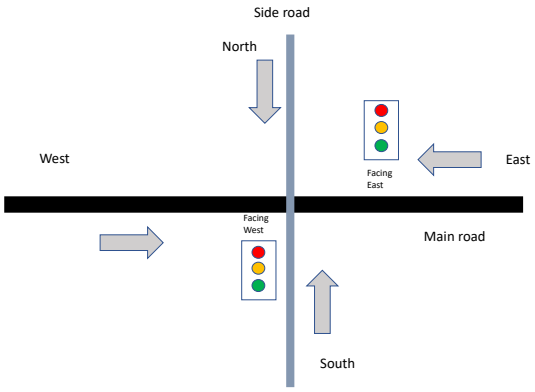

Figure 1 Traffic Light at Crossroads

(i) **(4 points)** Draw a UML state machine (also known as a statechart) which models the main road traffic light facing west (see Figure 1). Make sure to use UML events, conditions and actions correctly in your model. Your model should use a clock that generates a regular *tick* event after each unit time interval, and Boolean variables for the request button and car-sensor.

(ii) **(2 points)** Write a set of LTL trap properties which give 100% edge coverage for your model in answer to part (i).

(iii) **(3 points)** After just a few weeks, some drivers began to complain to the city planning authority that this traffic light system was unfair. Which drivers complained and what did they say? Use your answer to part (i) in your explanation.

(iv) **(2 points)** Using your answer to part (i), write down an LTL formula that models the unfairness situation which the drivers in part (iii) complain about.

(v) **(4 points)** Is it possible to model-based test an implementation of these traffic light requirements for fairness? Explain your reasoning in detail, using your answer to part (iv).

(Please turn over)

Question 4. **(Total 25 points)**

Harry Hopeful wanted to unit test the following code

```
myPoly(int x, y) {
      int result, w, i, j;

      result = 0;
      w = 0;
      i = 1;
      while (1 <= i <= y) {
            j = 1;
            while (1 <= j <= x) { w = w + 2; j++}
            result = result + w;
            w = w - x;
            i++;
      }
      return result;
}
```

(i) **(2 points)** Draw a condensation graph for Harry's code.

(ii) **(5 points)** Harry was particularly interested in testing the loops in this code, and was able to recall the graph theoretic concepts of a simple path and a prime path. Write out the set of all prime paths for the condensation graph that is your answer to part (i) above.

(iii) **(4 points)** For a given condensation graph *G*, we say that a glassbox control flow test requirement (i.e. a path in *G*)

$$\text{TRx} = p = v_1, \ldots, v_k$$

implies another control flow test requirement (i.e. a path in *G*)

$$\text{TRy} = q = v'_1, \ldots, v'_j,$$

if, and only if, every test case TC for *G* that satisfies TRx also satisfies TRy.

Looking over the prime paths of your answer to part (ii) as test requirements, write down all the pairs of prime paths (*p, q*) such that *p implies q*.

(iv) **(4 points)** Using your answers to parts (ii) and (iii), write out a minimal set of control flow test requirements to achieve 100% prime path coverage (PPC). Explain your answer.

(v) **(2 points)** Harry was told that the functional requirements for his code were the following JML pre and postconditions:

```
requires x >= 0 & y >= 0

ensures \result == 2*\old(x)*\old(y)
```

Using his test suite, Harry found that the code contained an error on just one single line of code. **(a)** Write down the line of code containing the error, and explain why this line is incorrect with respect to the JML requirement above? **(b)** How could you correct the program, so that it satisfies the JML requirement, by changing this single line only?

(vi) **(8 points)** To his surprise, Harry realized that only one prime path test requirement was guaranteed to reveal the single line coding error of part (v). All his other prime path test requirements were either irrelevant or could miss the error due to an unlucky choice of his satisfying test case. (Harry did not think himself a lucky tester!)

Which single prime path test requirement *tr* is guaranteed to find the error for any test case *tc* that is chosen to satisfy *tr*? Carefully explain your answer with reasoning about the code structure. What can you conclude about the ability of 100% PPC coverage to find errors in loops compared with other loop testing strategies that you know of?